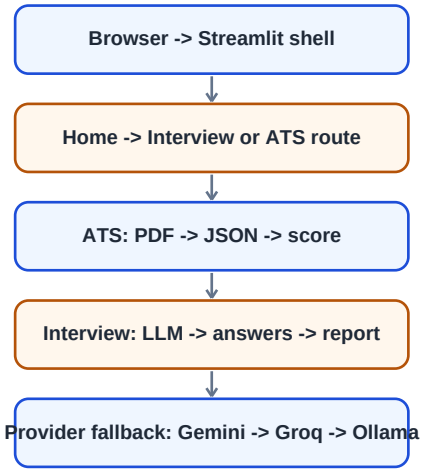
SmartHire

AI Career Tools: Deep Technical Project Analysis

Architecture, runtime flow, scoring model, LLM integration, operations, risks, and roadmap



Repository: /Users/paraspingale/Desktop/Projects/ATS_Interview

Review date: 11 October 2026

Prepared from direct source inspection and bounded local checks

1. Executive summary

SmartHire is a Streamlit application that combines two career-assessment tools behind one navigation shell: an ATS resume analyzer and an AI mock interview coach. The root app configures the page, applies shared styling, and exposes Home, AI Mock Interview, and ATS Resume Analyzer routes. The domain logic remains split into ATS and YourInterviewer packages, preserving direct execution paths for both features.

The two pipelines use different computational models. ATS analysis is local and deterministic after PDF text extraction: a selected domain loads a JSON keyword dictionary, the resume is lowercased, and each substring match adds a configured weight. Interview assessment is stateful and model-driven: an LLM creates questions, the user answers one at a time, another LLM call evaluates each answer, and a report is assembled and written to a local text file.

Current implementation status

Area	Observed state
Unified UI	Implemented in root app.py with st.navigation and three routes.
ATS analysis	Implemented with PyPDF2, 14 JSON domain dictionaries, word splitting, weighted keyword scoring, and tier assignment.
Interview flow	Implemented as setup -> interview -> report using st.session_state.
Model providers	Gemini first, Groq second, Ollama third; provider failures are combined into a user-visible error.
Persistence	Interview reports are local .txt files under YourInterviewer/reports; no database or user account layer.
Validation	Python compileall passed. Full UI and live provider behavior were not run because Streamlit is not installed in the active environment.
Security concern	The source contains fallback API credential literals. Revoke them and use environment variables or Streamlit secrets.

The project is best understood as a single presentation shell over two independent feature pipelines, rather than as one shared backend service.

2. Repository map and responsibility boundaries

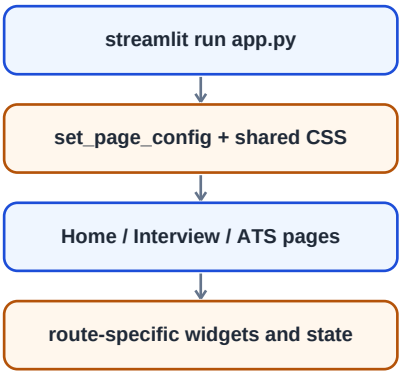
Path	Responsibility
app.py	Canonical Streamlit entry point, global page config, home cards, routes, and navigation.
ATS/TextExtraction.py	ATS page UI, PDF upload, PyPDF2 extraction, result rendering, and direct-entry compatibility.
ATS/ScoreCalculator.py	Domain-to-JSON mapping, JSON loading, weighted matching, and score tier selection.
ATS/Streamlit_chunking.py	Whitespace tokenization and extracted word count display.
ATS/*.json	Fourteen domain-specific keyword catalogs with weights, categories, and thresholds.
YourInterviewer/app.py	Interview UI, shared CSS, session state, question parsing, stage transitions, and report view.
YourInterviewer/Geminiapi.py	Prompt calls and fallback chain for Gemini, Groq, and local Ollama.
YourInterviewer/interview_agent.py	Evaluation prompt, score extraction, history, averages, and report text.
YourInterviewer/report_generator.py	Timestamped filename generation and local text-file persistence.
index.html	Older static launcher linking to localhost Streamlit routes; not used by the canonical root app.
modify_app*.py / test_groq*.py	One-off rewrite and manual provider probe scripts; not runtime dependencies.

Dependency surface

- Python runtime with Streamlit 1.57.0, PyPDF2 3.x, and requests 2.31.x according to requirements.txt.
- No database, queue, cache, authentication, container definition, CI workflow, or automated test suite is present in the repository tree.
- JSON and report paths are package-relative, which makes feature modules more portable than the root launcher alone.

3. Startup and navigation flow

The canonical entry point is app.py. It sets page metadata, imports the ATS renderer and interview main function, applies shared CSS, injects an animated background, creates three Streamlit Page objects, and runs st.navigation(...).



Home route behavior

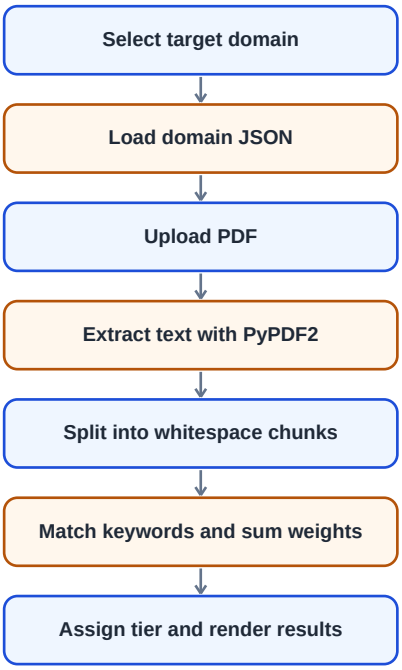
- Shows two cards: AI Mock Interview and ATS Resume Analyzer.
- Buttons call st.switch_page with the corresponding Page object.
- The route copy makes the product promise explicit: interview practice or domain-specific resume keyword coverage.

Compatibility behavior

- ATS/TextExtraction.py retains a direct execution branch and relative-import fallback.
- YourInterviewer/app.py supports package imports in the unified app and direct imports when run standalone.
- This compatibility layer is useful, but two import modes should be covered by smoke tests.

4. ATS resume analyzer: end-to-end flow

The ATS pipeline is simple and explainable. It does not use embeddings, an LLM, OCR, semantic similarity, or a resume parser. It extracts text from a PDF, splits on whitespace, and checks whether each configured keyword appears anywhere in the joined lowercased text.



Detailed execution

Step	Implementation detail	Output
Domain	selectbox exposes 14 domains; JsonSelctor maps names to files under ATS/.	Dictionary or error.
PDF	PyPDF2.PdfReader iterates pages and appends extracted text.	One full_text string.
Chunking	Chucking calls text.split() and displays token count.	List of chunks.
Matching	ScoreCalculator lowercases joined chunks and checks keyword substring membership.	Matched skill records.
Scoring	Each keyword match adds its configured weight once.	total_score.
Tiering	Thresholds are scanned; expert overflow becomes Expert+.	candidate tier.
UI	Two metrics plus matched keyword and weight list.	ATS Evaluation Results.

Scoring formula

For domain d and keyword set K: total_score = sum(weight(k) for k in K when lowercase(k) is a substring of lowercase(resume_text)). A keyword contributes once even if it appears many times.

Behavior edge cases

- Substring matching can create false positives, such as a short keyword inside a longer word.
- Scanned or image-only PDFs may yield little or no text because OCR is not implemented.
- Overlapping or synonymous skills are not normalized; each JSON key is independent.
- Thresholds vary by domain, usually placing fresher at 0-20 or 0-25 and expert at 46+ or 51+.

5. ATS knowledge base

The ATS dictionaries are configuration-driven JSON files. Each contains a domain identifier, a keywords object, and scoring_thresholds. The repository has 14 dictionaries with 10 to 14 keywords each. Adding a domain is mostly a data change, provided the selectbox and mapping are updated.

Domain family	Examples	Keyword count
Languages and frameworks	Python, Django, FastAPI, React, Java, Spring	10-14
Data and ML	pandas, NumPy, scikit-learn, TensorFlow, PyTorch	12-14
AI specializations	LLM, transformers, computer vision, NLP, reinforcement learning	12-14
Platform and delivery	Docker, Kubernetes, CI/CD, AWS, monitoring	10-13

Configuration contract

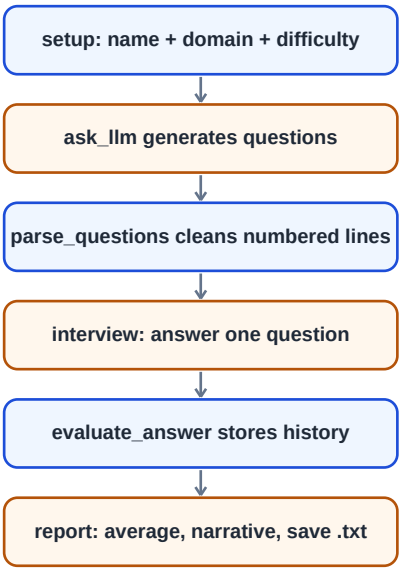
```
{
  "domain": "python_backend",
  "keywords": {
    "fastapi": {"weight": 5, "category": "framework"}
  },
  "scoring_thresholds": {
    "fresher": {"min": 0, "max": 20},
    "intermediate": {"min": 21, "max": 45},
    "expert": {"min": 46, "max": 100}
  }
}
```

Extension path

- Add a JSON file with the same schema.
- Add a display-name mapping in JsonSelctor.
- Add the display name to the ATS selectbox.
- Add fixtures for known resume strings, threshold boundaries, and missing-file behavior.

6. AI mock interview: state machine and data flow

The interview UI is a three-stage state machine held in Streamlit session state. Streamlit re-executes the script after form submissions, so the state fields preserve progress between reruns.



Session state fields

Field	Purpose
stage	setup, interview, or report.
questions_list	Parsed questions returned by the model.
current_q_index	Question currently displayed.
agent	MockInterviewAgent with history and totals.
domain	Free-text domain used for generation and evaluation.

Question generation

ask_llm asks for 15 questions at the selected difficulty in a strict numbered-list format. parse_questions removes a leading number and optional asterisks, then keeps lines longer than ten characters. It does not enforce exactly 15 questions or validate content quality.

7. Answer evaluation and report generation

For each non-empty answer, MockInterviewAgent.evaluate_answer builds a structured prompt containing the student name, topic, question, and answer. It asks for a score, evaluation, missing points, improved answer, and one practical suggestion. The raw response is retained with a numeric score.

Score extraction

extract_score scans lines containing the word score, replaces slash and colon characters with spaces, and returns the first integer from 0 to 10. If the model deviates from the requested format, the score becomes 0. The app continues, but malformed responses can be silently under-scored.

Final report calculation

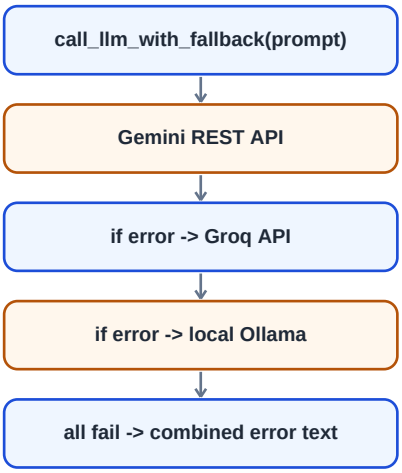
- total_questions is the length of history.
- average_score = total_score / total_questions, with zero for an empty history.
- Narrative bands are Excellent (8+), Good (6+), Average (4+), or Needs improvement.
- Every question is printed with topic, answer, score, and full LLM evaluation.
- save_report writes a timestamped text file under YourInterviewer/reports.

Persistence model

Concern	Current behavior	Effect
Format	Plain text .txt	Readable but not machine-friendly.
Location	Package-relative reports directory	Works locally; no shared storage.
Identity	Filename uses student name and timestamp	Personal data may be exposed in filenames.
Concurrency	No locking or central index	Not suitable for multi-user history.
Retention	No cleanup policy	Reports accumulate indefinitely.

8. Provider integration and fallback chain

Geminiapi.py centralizes model traffic behind call_llm_with_fallback. Question generation and answer evaluation use the same chain, allowing the interview to continue if a preferred provider is unavailable.



Provider	Transport	Timeout	Role
Gemini	requests.post to Google Generative Language API	30 sec	Primary cloud model.
Groq	requests.post to OpenAI-compatible endpoint	30 sec	Cloud fallback.
Ollama	requests.post to localhost:11434/api/generate	120 sec	Local fallback.

Failure handling

- Non-200 responses and exceptions become strings beginning with an error marker.
- The next provider is attempted when the previous result starts with that marker.
- If all providers fail, the setup stage displays a combined error summary.
- There is no retry backoff, circuit breaker, provider health cache, or structured error type.

Security finding

Geminiapi.py and manual Groq probe scripts contain fallback credential literals. This report intentionally does not reproduce them. Treat them as compromised: revoke or rotate them, remove the literals, and load credentials only from environment variables or Streamlit secrets. The current os.getenv calls are the right direction, but defaults defeat that protection.

9. UI and presentation layer

The application uses Streamlit widgets for interaction and custom CSS for a branded experience. YourInterviewer/app.py defines the shared visual system: Inter font, warm neutral background, amber gradient headings, rounded cards, animated blurred orbs, and an inline SVG technology banner. The root app reuses this styling for the integrated shell.

Surface	Observed behavior
Home	Two equal cards with CTA buttons and hover elevation.
Interview setup	Centered form for name, free-text domain, and difficulty.
Interview question	Question card and text area; empty submissions are rejected.
Report	Completion message, code block with report text, and local path info.
ATS results	Two metrics for score and tier, followed by matched keyword list.

Legacy static launcher

index.html is a separate dark-themed static page linking to localhost:8501/interview and /ats-analyzer. It is useful as a visual prototype or external launcher, but the root Streamlit app is the canonical integrated entry point.

Visible UX gaps

- No explicit upload size limit or page-level error handling around malformed PDFs.
- Interview domain is free text while ATS domain is controlled, so naming conventions can diverge.
- The report is shown in a code block and is not offered as a download.
- Injected CSS depends on Streamlit DOM test IDs that can change across versions.

10. Runtime, deployment, and operations

The project is designed for a local or single-process Streamlit deployment. The shortest launch is streamlit run app.py. There are no Docker, process manager, reverse proxy, or cloud deployment manifests in the repository tree.

Concern	Current design	Implication
Configuration	Environment variables are read, but credential fallbacks remain.	Secrets must be rotated and injected securely.
Storage	Local text reports.	Ephemeral containers lose reports without a volume.
PDF processing	In-memory upload and extraction.	Large or malformed files need limits and errors.
Model calls	Synchronous requests in callbacks.	Latency blocks the user session.
Observability	Print statements only.	No structured logs or provider metrics.
Testing	No formal suite in the tree.	Regression risk is high.

Recommended production boundary

- Keep Streamlit as the presentation layer, but move model calls and persistence behind service modules with structured return types.
- Use secrets management and fail clearly when required configuration is missing.
- Add timeouts, retry policy, provider telemetry, and user-friendly error states.
- Add route smoke tests, ATS tests, parser tests, and deterministic score parsing tests.

11. Verification performed for this review

This report is based on direct source inspection and bounded local checks. No code was changed as part of the analysis.

Check	Result
Repository inventory	36 non-cache files found, including 14 ATS JSON dictionaries and core Python modules.
Python syntax	python3 -m compileall -q app.py ATS YourInterviewer passed.
Static review	Routes, imports, state, provider chain, scoring, reports, and compatibility branches inspected.
ATS runtime smoke test	Blocked because importing a Streamlit-dependent module raised ModuleNotFoundError in the active environment.
Live Streamlit route check	Not run because Streamlit is not installed in the active environment.
Live provider behavior	Not run; requires credentials, network, and/or local Ollama.
PDF output QA	This document is rendered and checked separately after generation.

Evidence basis

- app.py and YourInterviewer/app.py define navigation and stage flow.
- ATS/TextExtraction.py, Streamlit_chunking.py, and ScoreCalculator.py define the PDF and scoring path.
- ATS/*.json defines the domain knowledge base.
- Geminiapi.py, interview_agent.py, and report_generator.py define provider integration and report behavior.
- requirements.txt defines declared runtime dependencies.

12. Prioritized improvement roadmap

Priority	Change	Why it matters
P0	Remove and rotate embedded API credentials; use secrets only.	Prevents unauthorized model usage.
P0	Add PDF parsing errors and file size/page limits.	Protects the main user path.
P1	Add tests for score boundaries, matching, parser behavior, and averages.	Locks down core computations.
P1	Replace substring matching with normalized token/phrase matching and optional synonyms.	Reduces false positives.
P1	Make question count and provider responses structured JSON.	Avoids brittle line parsing and silent zero scores.
P2	Persist reports in a database or object store with retention.	Supports multi-user deployment.
P2	Add downloads and remove personal data from filenames.	Improves UX and privacy.
P2	Add deployment artifacts, health checks, logs, and metrics.	Makes the app operable beyond a laptop.

Final assessment

SmartHire already has a coherent demo slice: one branded entry point, two understandable user journeys, data-driven ATS configuration, and a complete interview loop from question generation to saved report. Its main technical risks are LLM output reliability, provider credential security, local-only persistence, and lack of automated verification. Those risks are isolated enough that the project can be hardened incrementally without redesigning the user-facing concept.

End of report